ADAPTIVE SUPPLY CHAIN INTELLIGENCE

An Integrated Agentic-RAG Platform for the South India Automotive Logistics Corridor

Live Event Intelligence • Corridor Risk Graph • Agentic Retrieval & Adaptive Computation

Final Project Design & Implementation Document — Version 2.0 (Integrated Edition)

Document item	Value
Industry focus	Automotive & auto-components logistics (containerised components, import and export)
Region focus	South India port–hinterland corridor: Karnataka, Tamil Nadu, Andhra Pradesh, Kerala
Core architecture	One pipeline over one shared state & knowledge layer; agentic RAG controller; deterministic optimisation and layered verification
Decision mode	Decision support with mandatory verification, cited evidence and human review
Team size	5 members
Version / date	2.0 — 16 October 2026 (revises v1.0 dated 30 September 2026)

Reference basis

- Adaptive Supply Chain Intelligence — Technical Design and Methodology Report v2.0 (39 pages): remains the authoritative reference for algorithms, schemas and the evaluation protocol.
- Adaptive Supply Chain Intelligence — Final Project Document v1.0 (the document this version revises).
- Adaptive Computational Intelligence concept; Experiential Learning Phase I–III proposal (Autonomous Supply Chain Event Interceptor, Agentic RAG).

This document is a project specification, not a results report. Every number marked "target", "proposed" or "illustrative" is a plan or an example to be validated, not an achieved result.

0. What changed from v1.0

This revision responds to four review requests. The table maps each request to the design change and to the section where it is specified.

Request	What changed	Section
1. Complete integration, not independent modules	The three “modules” are now stages of one pipeline over a single Unified State & Knowledge Layer (USKL) with one canonical entity registry. Every store (graph, events, documents, time-series, decisions) uses the same node/edge IDs. Four closed feedback loops make later runs depend on earlier ones. The software-supply-chain / GitHub-dependency idea has been removed from the design; GitHub appears only as CI tooling.	3, 4, 5
2. Limit scope to one industry and region, based on available data	Scope fixed to automotive & auto-components logistics in the South India port–hinterland corridor. A dataset matrix lists each public source, what it supplies, and where synthetic data is unavoidable. A Week-2 data-readiness gate verifies access before design is frozen.	2
3. Computational RAG made agentic	Retrieval is no longer a single lookup. An agentic RAG core plans, chooses among text, graph and time-series retrieval *and* computation tools, grades evidence sufficiency, rewrites queries, cites sources, and stops under a cost-aware policy. It is compared against no-RAG, fixed-RAG and LLM-planner baselines.	6
4. Other improvements	Real-event backtesting; a plant-level “time-to-line-stop” KPI suited to just-in-time automotive; PortWatch anomaly-triggered events; tiered MVP; retrieval-specific evaluation; retrieval-poisoning defences; calibrated uncertainty; updated roadmap built on vertical slices.	10–13

1. Executive summary

The system is an adaptive supply-chain intelligence platform for the **South India automotive logistics corridor**. It turns live disruption information (cyclone and heavy-rain alerts, port congestion, road closures, shipping schedule changes) into a verified, evidence-cited response plan: which paths, ports and plants are affected, how severely, how long plant inventory will last, and which rerouting plan remains feasible under uncertainty.

Its defining property is **integration**. A disruption event is resolved against the same graph that the optimiser uses; the same graph IDs tag the documents that the retriever searches; retrieved analogues set the scenario priors; realised port data later recalibrates the graph; and every plan is stored in a memory that the next run retrieves. No component can be demonstrated meaningfully in isolation.

The central AI contribution is an **agentic RAG controller** that learns how much retrieval and computation a given event deserves. It decides, step by step, whether to retrieve more evidence, compute exposure, propagate effects, generate scenarios, optimise, verify or stop, under an explicit compute-cost objective and hard safety rules. The LLM interprets and explains; deterministic code optimises and verifies.

> **Scope in one line**
> Containerised auto-component flows between plant clusters (Bengaluru–Hosur, Sriperumbudur–Oragadam, Sri City, Coimbatore) and gateway ports (Chennai, Kamarajar, Kattupalli, Krishnapatnam, Tuticorin, Cochin, New Mangalore), with rail and road links, on a daily planning horizon.

2. Industry & region scope

2.1 Why automotive logistics in South India

- **Just-in-time sensitivity.** Automotive plants hold days, not weeks, of inbound components. A port or road disruption translates quickly into a possible line stoppage, which gives the project a concrete, plant-level outcome metric (Section 5.3).
- **Dense, short corridors with real alternatives.** Several plant clusters sit within a few hours of more than one gateway port, with rail links from inland container depots. Rerouting decisions are therefore genuine, not trivial.
- **Recurring, documented regional hazards.** The north-east monsoon and Bay of Bengal cyclones regularly disrupt the Chennai-area coast. This yields a steady stream of events for live demonstration and a historical case library for retrieval.
- **Open data availability.** Port-level activity, hazard alerts, news events, road/rail geometry and trade statistics are public (Section 2.4). Plant-level flows are not, and are generated synthetically.

2.2 Scope boundary

In scope	Out of scope
Industry: automotive & auto-component logistics only	Other industries; pan-India or global network
Region: Karnataka, Tamil Nadu, Andhra Pradesh, Kerala corridor; out-of-region gateways only as boundary nodes	Full national network; international hinterlands
One commodity class: containerised auto components, inbound and outbound	Multi-commodity optimisation; roll-on/roll-off finished vehicles (Tier 3 stretch only)
Road, rail and port nodes/edges; daily horizon	Air freight; last-mile delivery; tier-3+ supplier mapping
Events: weather/hazard, port congestion or strike, road/rail closure, shipping schedule change, trade-policy change, plant outage	Autonomous execution of logistics actions; replacing planners
Synthetic benchmark + real-event backtests + live demonstration	Claiming real-world forecasting accuracy from live news

2.3 Corridor graph size

Target size: roughly **100–150 nodes and 300–500 edges**: about 7–8 ports, 5–6 rail terminals/inland container depots, 25–40 plant or supplier nodes grouped by cluster, 15–25 warehouse/freight-station nodes, and 10 or so outbound market gateways. This is large enough for non-trivial path enumeration and small enough for fast optimisation and clear visualisation.

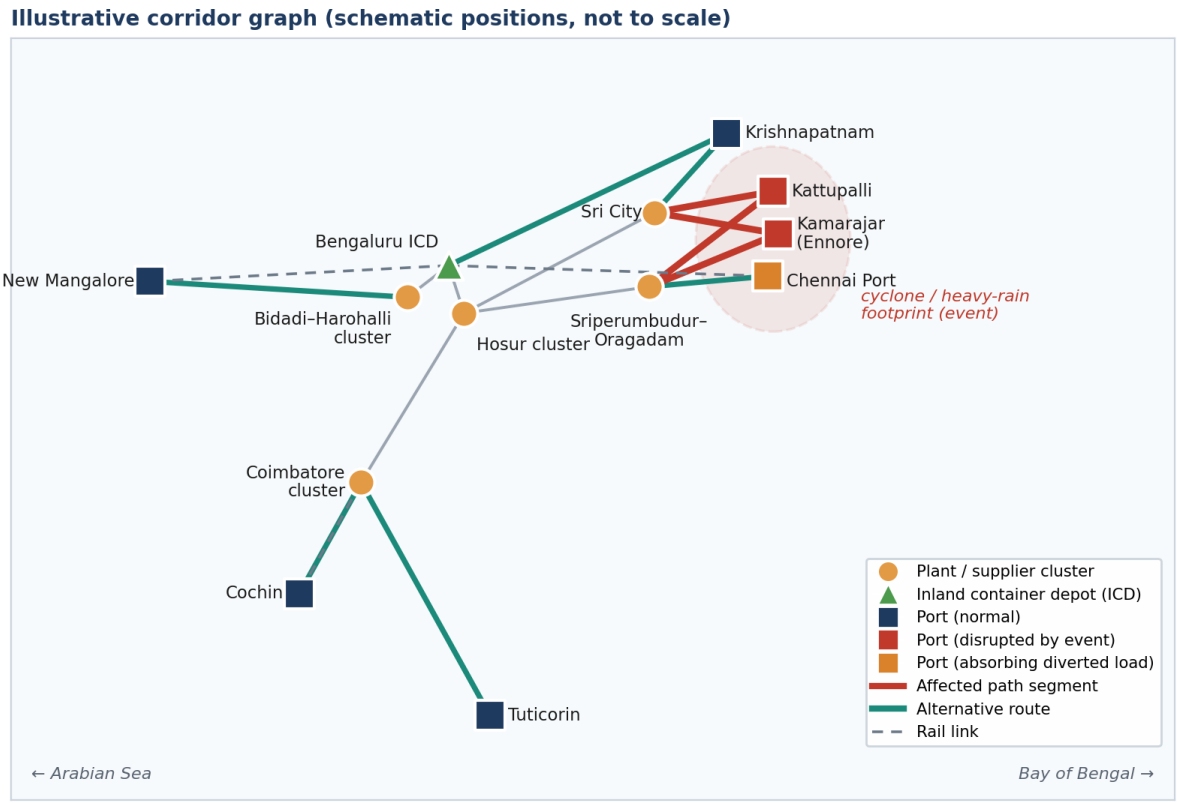


Figure 1. Illustrative corridor graph with an event footprint, affected path segments and alternative routes.

2.4 Dataset matrix

Each source is mapped to a system function and a real-versus-synthetic status. Access terms, rate limits and coverage **must be verified in Weeks 1–2** (gate G0); the fallback column states what the project does if a source fails.

Source	Provides	Role in the system	Status / fallback
IMF PortWatch (free, open)	Daily AIS-derived port calls and trade-volume estimates; published with a multi-day lag	Port baseline throughput; ground truth for backtests; anomaly-triggered events	Real. Fallback: synthetic port series calibrated to published totals
GDACS (via PortWatch/HDX or direct)	Alerts for cyclones, floods, earthquakes with location and severity	Live hazard events; hazard-to-port intersection	Real. Fallback: cached alerts
GDELT and news/port-authority RSS	Global news events; local advisories	Unstructured event text for extraction; documents for vector memory	Real, noisy. Fallback: curated article set
Weather forecast API (e.g. Open-Meteo) and IMD bulletins	Rainfall and wind forecasts; official warnings	Severity priors; time-series retrieval	Real. IMD may need manual or scraped bulletins
OpenStreetMap (via OSMnx)	Road and rail geometry; port and terminal locations	Edge topology, distances, transit-time estimates	Real
UN Comtrade / India trade statistics; Ministry of Ports and data.gov.in	Auto-sector trade value by partner; port traffic statistics (granularity to be confirmed)	Demand weights for outbound lanes; calibration of synthetic flows	Real, coarse. Fallback: published annual aggregates

Source	Provides	Role in the system	Status / fallback
Public lists of container terminals / ICDs and industrial clusters	Locations and names	Node set for rail terminals and plant clusters	Real locations; capacities synthetic
Plant inventory, supplier tiers, shipment flows, costs	Not public	Days-of-cover, flows, edge costs, demand	Synthetic, generated and calibrated so totals match port and trade aggregates

> **Honest data position**
> Topology, locations, hazards, news and port activity are real. Plant-level operational data is synthetic by necessity. Results on the synthetic benchmark measure the **method**; backtests on real events (Section 11) check that the method behaves sensibly on real signals. Neither is claimed as proof of real-world forecasting accuracy.

3. Integrated architecture

3.1 From modules to one system

In v1.0 the three components could be read as separately buildable units. In this version each is a stage of one pipeline and cannot produce a meaningful output without the others. The dependency table makes the coupling explicit and is used as an integration test checklist.

Stage	Consumes (from)	Produces (for)	Breaks if missing
A. Event Intelligence	Feeds, analyst input, PortWatch anomalies; graph registry for entity resolution; vector memory for disambiguation and priors	Typed event with graph-resolved targets (to B, C)	Events cannot be grounded to graph IDs
B. Corridor Risk Graph	Resolved event (A); time-series baselines; retrieved severity/duration priors (C)	Affected paths, exposure, line-stop risk, alternative capacity (to C, optimiser)	No path impact, no features for the controller
C. Agentic RAG Controller	Event (A); graph features (B); all knowledge stores; cost budget λ	Operator sequence, evidence set, scenarios, explanation (to optimiser, verifier, dashboard)	No adaptive depth, no evidence, no explanation
D. Optimise & Verify	Graph state (B); scenarios (C); event grounding (A)	Verified plan or escalation (to E)	No safe plan release
E. Dashboard & Human Review	Plan, paths, evidence, trace	Decisions, edits, outcomes (back to memory, graph, controller training)	Loop never closes; system cannot learn

3.2 Unified State & Knowledge Layer (USKL)

All components read and write one layer made of five stores that share a **canonical entity registry**. A node such as `PORT_CHENNAI` has the same ID in the graph, in event targets, in document metadata, in time-series keys and in plan reservations. This is the main integration mechanism.

Store	Technology (MVP)	Content	Linked by
Corridor Graph	NetworkX, persisted to SQLite/JSON	Nodes, edges, capacities, multipliers, plant days-of-cover	Entity registry IDs

Store	Technology (MVP)	Content	Linked by
Event Store	SQLite	Typed events, clusters, lifecycle state (new, updated, retracted, expired)	Target node/edge IDs
Vector Memory	FAISS + sentence-transformers	Advisories, incident reports, past event summaries, playbooks; each chunk tagged with graph entity IDs, region, hazard type and date	Entity tags in metadata
Time-series Store	DuckDB / SQLite	Port calls, rainfall, baselines, realised outcomes	Port/node IDs
Decision & Trace Memory	SQLite + JSONL	Human approvals/edits, outcomes, operator traces, plan history	Event, plan and entity IDs

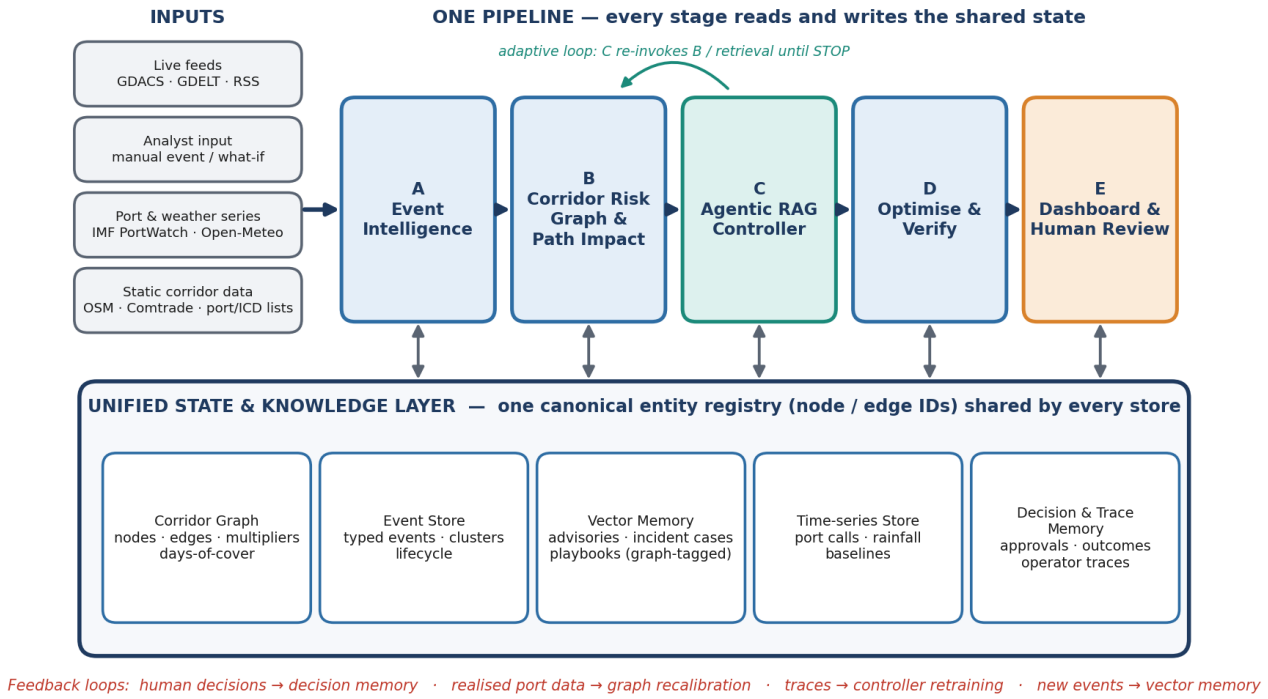


Figure 2. Integrated architecture: one pipeline over one shared state and knowledge layer.

3.3 Pipeline logic

1. A live source, a PortWatch anomaly or an analyst submits an event.
2. Event Intelligence extracts a typed record and resolves it against the entity registry, using retrieval to disambiguate places and set priors.
3. The graph engine applies the event to the graph and computes direct exposure, affected paths and plant line-stop risk.
4. The agentic controller observes the combined state and chooses the next retrieval or computation operator; each result is written back to the state.
5. The deterministic optimiser produces the candidate plan; the three-layer verifier checks grounding, feasibility, decision sanity and citation support.
6. The dashboard presents plan, affected paths, evidence and trace for human approval, edit or rejection.
7. The decision, and later the realised outcome, are written to memory and to the time-series store.

3.4 Closed feedback loops

Loop	Flow	Effect on later runs
L1 Outcome → graph	Realised PortWatch activity after an event updates edge/node multiplier estimates and severity priors	Better direct-impact estimates for the same hazard type and port
L2 Decision → memory	Planner approvals, edits and rejections are stored with the event and plan	Retrieved as decision precedents; used in plan-churn checks
L3 Trace → controller	Operator traces with measured costs become oracle-labelled training data	Controller retraining; regret-versus-cost improvement
L4 Event → knowledge	Every validated event and its resolution is chunked, tagged and added to vector memory	Richer analogue retrieval for future events

3.5 Integration contracts and rules

- Every operator follows `run(state, cfg, rng) → (new_state, OpResult)`. State is typed (Pydantic v2) and immutable at operator boundaries; randomness only through `rng`; cost is measured, never self-reported.
- **Walking skeleton first.** By end of Week 2 an event travels end to end (event → graph → rule router → optimiser → verifier → dashboard stub) with stubs where needed. Every later week replaces stubs with real components while the end-to-end test stays green.
- **Integration test in CI:** a scripted event must produce a verified plan with a complete trace on every merge. A change that breaks it cannot merge.
- Mocks are temporary scaffolding; none may remain at the Week-13 integration freeze.

4. Stage A — Live Event Intelligence (corridor-specific)

This stage converts changing external information into typed, graph-grounded events. Perception always runs; adaptivity applies only to the expensive downstream operators.

4.1 Event sources and processing

Step	Implementation	Integration point
Ingest	Scheduled polling of GDACS, GDELT and RSS; analyst event form; PortWatch anomaly detector that raises an event when a port's daily calls fall well below its baseline	Time-series store feeds the detector; events are stored in the event store
Classify	Rules or a lightweight LLM classifier for corridor relevance	Uses region/hazard tags from the registry
Extract	Rules first; structured-output LLM as fallback. The extraction LLM has no tools and sees only the article text	Output validated against Pydantic schema
Resolve	Fuzzy matching and embeddings against the graph gazetteer (node names, aliases, geography); retrieval of similar past resolutions	Targets expressed as registry IDs with probabilities
Validate	Schema and grounding checks; source corroboration count	L1 verification record
Store	Deduplication, updates, retractions, expiry; cluster into a belief state	Event store; chunk and tag into vector memory (loop L4)

4.2 Regional event taxonomy

Event type	Typical regional example	Graph effect
HAZARD_PORT	Cyclone landfall or red rain alert near Chennai–Ennore coast	Port capacity multiplier, delay at affected ports
HAZARD_ROAD	Flooded underpass or highway closure on a plant–port road	Edge closure or multiplier, added transit time
PORT_CONGESTION / STRIKE	Berth congestion, labour action, customs system outage	Port handling capacity, dwell time
RAIL_DISRUPTION	Track blockage on an ICD–port rail link	Rail edge multiplier; shift of load to road
SHIPPING_SCHEDULE	Blank sailings or route diversion on a trade lane	Outbound gateway capacity or delay
TRADE_POLICY	Tariff or regulatory change affecting an export market	Demand weight on outbound lanes
PLANT_OUTAGE	Plant or supplier shutdown	Node capacity, upstream flow

Example structured event


```
{"type":"HAZARD_PORT","targets":[{"id":"PORT_KAMARAJAR","p":0.92},{"id":"PORT_KATTUPALLI","p":0.8
8}],
 "capacity_multiplier":0.40,"onset":"2026-10-20","duration_days":3,
 "confidence":0.78,"sources":["gdacs:...","rss:..."],"evidence_ids":["doc_0412","doc_0977"]}
```


5. Stage B — Corridor Risk Graph & path-impact prediction

5.1 Graph model

The corridor is a directed graph G=(V,E). Nodes: suppliers/plants, ports, inland container depots, warehouses and freight stations, outbound market gateways. Edges: road, rail and port-to-gateway links. Each element stores capacity, utilisation, cost, transit time and a current multiplier; plant nodes also store **inventory days-of-cover**. The graph is built from OpenStreetMap geometry and public terminal/port lists, with synthetic operational attributes calibrated to public aggregates.

5.2 Affected-path algorithm

1. Resolve event targets to registry IDs (from Stage A).
2. Apply the multiplier or delay and build the affected k-hop subgraph.
3. Enumerate feasible plant-to-gateway paths traversing affected elements.
4. Compute path exposure, alternative capacity, bottleneck concentration and criticality.
5. Compute plant-level line-stop risk (Section 5.3).
6. Run propagation if the controller selects it; rank paths by risk; write features to the shared state.

5.3 Risk features and the line-stop metric

Feature	Definition	Used by
Exposure	Share of baseline flow behind the affected element	Controller, dashboard
Criticality	Normalised betweenness + utilisation + demand dependency	Dashboard, controller
Alternative capacity ratio	Spare capacity on substitute routes and ports	Optimiser, scenarios
Bottleneck concentration	Whether important paths share few assets	Controller
Uncertainty	Event confidence + propagation ensemble spread + evidence sufficiency	Controller

Feature	Definition	Used by
Line-stop risk (plant)	Probability, across scenarios, that expected inbound delay exceeds the plant's days-of-cover; also reported as time-to-line-stop	Controller, optimiser objective, dashboard

`PathRisk(p) = Exposure × Criticality × Severity × UncertaintyFactor × Concentration` is an engineering ranking score, not a calibrated probability. Line-stop risk is the headline domain KPI because it is what a planner in a just-in-time plant ultimately cares about.

5.4 Propagation

Tier 1 uses a deterministic hop-decay cascade. Tier 2 adds a load-redistribution simulator (port capacity down → diverted load up → road congestion up → inventory down → delay up) that also serves as ground truth for the synthetic benchmark. A GNN surrogate is **Tier 3** and is retained only if it beats the heuristic in evaluation. Retrieved analogue events supply priors for congestion and recovery time (Section 6.6).

6. Stage C — Agentic RAG computational core

6.1 Concept

In a conventional RAG pipeline, retrieval is a fixed step before generation. Here it is an **agentic, computational** process: a controller observes the state, decides what to do next, calls a tool, grades the result and decides again. "Computational" means that tools include not only document retrieval but also graph queries, time-series queries and numerical computation (path analysis, propagation, scenarios, optimisation). Retrieval and computation are treated as comparable operators, each with a measured cost, so the system can learn when another retrieval is worth more than another computation.

> **Design stance**
> One orchestrator with typed tools, not a swarm of autonomous agents. This keeps state, cost accounting and safety masks tractable for a five-person team.
>
> The LLM interprets text, rewrites queries, grades evidence and writes explanations. It never decides feasibility or optimises; deterministic code does.

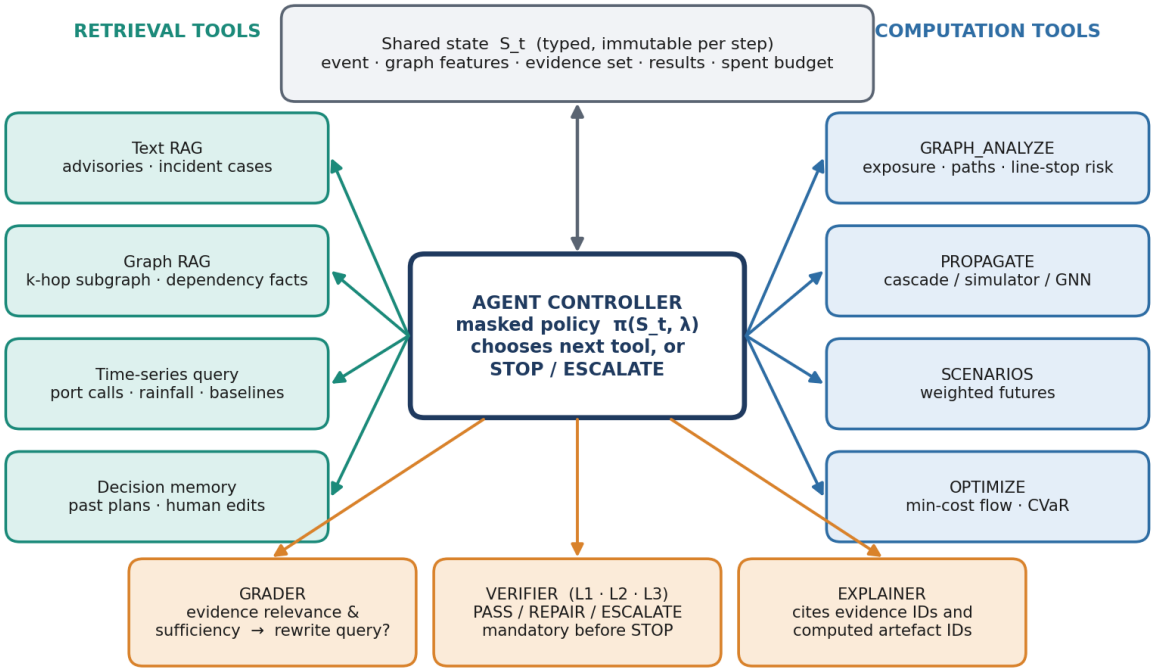


Figure 3. Agentic RAG loop: the controller selects retrieval or computation tools over shared state.

6.2 Retrieval tools (four sources, one registry)

Tool	Store	Query form	Returns	Used for
RETRIEVE_TEXT	Vector memory	Semantic query + filters (entity IDs, hazard type, date range)	Ranked passages with IDs, dates, sources	Analogue incidents, advisories, playbook steps
RETRIEVE_GRAPH	Corridor graph	k-hop subgraph around entity IDs; dependency facts as short structured summaries	Subgraph + natural-language facts with provenance	Dependency context for explanation and priors
RETRIEVE_TS	Time-series store	Parameterised query (port, window, hazard analogue)	Series statistics: baseline, drop, recovery time	Severity/duration priors; outcome checks
RETRIEVE_DECISIONS	Decision & trace memory	Similar event + plan lookup	Past plans, edits, outcomes	Precedent and plan-churn context

6.3 Agent roles

Role	Implemented by	Responsibility
Controller (planner)	Learned masked policy (MLP); rule router and LLM planner as baselines	Choose next operator from the valid set; decide STOP or ESCALATE
Query rewriter	LLM, constrained to structured output	Reformulate a failed query using entity IDs and hazard terms; limited to 2 rewrites per query
Retrievers	Deterministic code over the USKL stores	Execute queries, return IDs and provenance
Grader	Small LLM or cross-encoder + rules	Score relevance of each item and sufficiency of the evidence set (0–1); triggers rewrite, more retrieval or proceed

Role	Implemented by	Responsibility
Computation tools	Deterministic code (graph, simulator, scenarios, OR-Tools)	Produce numerical artefacts with IDs
Verifier	Deterministic checks (+ citation checker)	PASS, REPAIR or ESCALATE
Explainer	LLM over computed artefacts and evidence only	Write the rationale; every claim must cite an evidence ID or artefact ID

6.4 Action space

Operator	Cost class	Mask rule (valid when)
GRAPH_ANALYZE	Low	Event resolved; always valid first after perception
RETRIEVE_TEXT / _GRAPH / _TS / _DECISIONS	Low–medium	Event resolved; per-tool call limit not exceeded
REWRITE_QUERY	Low	Previous retrieval graded insufficient; rewrite count < 2
GRADE	Low	New evidence exists since last grade
PROPAGATE	Medium	Graph analysis done
SCENARIOS	Medium	Graph analysis done; uncertainty or impact above gate
OPTIMIZE	High	Required state available (graph analysis; scenarios if requested)
VERIFY	Low	A plan exists
STOP	None	Only after VERIFY = PASS
ESCALATE	None	Always valid; forced by safety rules

6.5 Control loop

At each step the controller observes a feature vector built from the shared state: event confidence and severity, exposure, line-stop risk, evidence sufficiency, number and age of retrieved items, operators already run, steps and cost spent, and the budget parameter λ. It selects the next valid operator; the result is written back immutably; the loop ends on verified STOP, ESCALATE, or the step limit (pseudocode in Appendix B).

Objective: $J(\pi) = E[\ \mathrm{Regret}(\mathrm{Plan}) + \lambda \cdot \Sigma\ \mathrm{Cost}(\mathrm{operator})\ ]$. Cost is measured wall-clock time plus LLM tokens per operator, profiled once and logged per run.

6.6 How retrieval is woven into the other stages

Consumer	What retrieval supplies	Effect
Stage A — entity resolution	Similar past place-name resolutions; gazetteer aliases	Fewer mis-grounded targets
Stage A/B — severity & duration	Distribution of impact and recovery times from analogue incidents and port-call series	Informed priors when the news text gives no duration
Scenario generation	Analogue outcomes → scenario set and weights	Evidence-based scenario probabilities, not arbitrary ones
Explanation	Cited passages and artefacts	Auditable rationale

Consumer	What retrieval supplies	Effect
Verification (L3)	Citation-support check: each cited ID must exist and support the claim	Blocks unsupported statements

6.7 Learning strategy

1. **Baselines first:** always-full pipeline; rule router (thresholds on confidence/exposure); fixed RAG (always retrieve once, then fixed order); LLM planner (ReAct-style, prompt-only).
2. **Planted-analogue benchmark:** the synthetic generator creates a historical case library in which some past events are true analogues of test events and carry known outcomes. This makes the value of retrieval measurable: a run that retrieves the right analogue should achieve lower regret.
3. **Oracle labels:** enumerate valid operator sequences (including retrieval steps) on training instances, compute realised regret and measured cost, and label the best sequence per λ.
4. **Imitation learning:** train a small masked MLP on oracle labels; add DAgger only if time permits; consider policy-gradient refinement only after the supervised system works.
5. **Honest reporting:** the rule router is a real baseline. If the learned controller only ties it, that is reported as is.

6.8 Safety for agentic RAG

- **Retrieved text is untrusted data.** It is wrapped as quoted evidence, never concatenated into instructions; the extraction LLM has no tools; only deterministic code, after L1 validation, may modify the graph.
- **Tool allow-list and hard masks;** step limit (e.g. 12), per-tool call limits and a budget cap. Hitting any limit forces ESCALATE with partial evidence.
- **Retrieval poisoning and staleness:** each chunk carries source, date and trust tier; low-trust or expired items are down-weighted and flagged; conflicting sources raise an uncertainty feature.
- **Citation-required output:** the explainer cannot release a claim without a supporting ID; L3 verification rejects unsupported claims.
- **Confidence gates:** high-impact and low-confidence events, out-of-distribution states and low policy confidence trigger the full-analysis fallback or human escalation.

7. Optimisation & verification

The plan is computed by deterministic network-flow optimisation (OR-Tools; SciPy HiGHS as an independent cross-check). Modes: deterministic when the event is clear and stable; expected-cost when scenario weights exist; CVaR/robust when tail risk matters; human escalation when checks fail. The objective includes unmet-demand penalties and a **plant line-stop penalty** tied to days-of-cover.

Layer	Checks	Action on failure
L1 Input	Schema, graph grounding, source corroboration, entity validity, evidence provenance	Reject / repair / escalate
L2 Plan	Capacity, flow conservation, closed edges, objective recomputation by a second solver	Repair or escalate
L3 Decision	Unmet demand, line-stop risk threshold, cost sanity, plan churn, confidence gate, citation support of the explanation	Repair or human confirmation

Targets from the methodology (≥99% catch rate for injected plan faults; unchanged state for injected-instruction cases) are goals to validate, not achieved results.

8. End-to-end demonstration scenario

Scenario (illustrative): during the north-east monsoon season, a red-level cyclone/heavy-rain alert covers the Chennai–Ennore coast. Reports indicate Kamarajar and Kattupalli will operate at about 40% capacity for three days, with flooding risk on plant–port roads. The values below are **illustrative** and show the intended behaviour, not measured results.

Step	Operator	Observation written to state (illustrative)	Cost
0	Perception (always)	GDACS alert + 2 news items → HAZARD_PORT at Kamarajar and Kattupalli, multiplier 0.40, 3 days, confidence 0.78; targets resolved to registry IDs	Low
1	GRAPH_ANALYZE	14 plant-to-gateway paths affected; ~38% of outbound corridor flow behind the two ports; Oragadam cluster time-to-line-stop ≈ 2.1 days	Low
2	RETRIEVE_TEXT	Three analogue cyclone-closure reports retrieved; duration prior 2–5 days; evidence sufficiency 0.55 (below 0.70 gate)	Medium
3	REWRITE_QUERY → RETRIEVE_TS	Query refined with port IDs; port-call series for two past analogues show a sharp drop and several-day recovery	Medium
4	GRADE	Sufficiency 0.82 ≥ 0.70 → proceed	Low
5	PROPAGATE	Diverted load raises Chennai Port utilisation; Krishnapatnam spare capacity can absorb a large share	Medium
6	SCENARIOS	Three weighted scenarios (moderate, severe, full closure) with analogue-informed weights	Medium
7	OPTIMIZE	CVaR plan: pre-reserve Krishnapatnam and Chennai slots; shift selected flows to rail from the Bengaluru ICD	High
8	VERIFY	L1, L2, L3 PASS; all explanation claims cite evidence or artefact IDs	Low
9	EXPLAIN → STOP	Plan, evidence and trace released for human approval	Low

Contrast case: a minor, high-confidence, low-exposure event (e.g. a short delay at a rarely used terminal) lets the controller skip retrieval, scenarios and propagation and go straight from graph analysis to a deterministic plan, which is where compute is saved.

After the event: realised PortWatch data for the affected ports is ingested (loop L1), the planner's decision is stored (L2), the trace becomes training data (L3), and the event summary joins vector memory (L4).

9. Dashboard

A decision-support command centre, not a chat window. React + Cytoscape.js with geo-positioned nodes.

View	Information
Event feed	Events, sources, confidence, lifecycle, PortWatch anomaly flags
Corridor map	Nodes/edges, affected paths, multipliers, flow (edge width), uncertainty (ring/opacity)
Risk panel	Ranked paths, exposure, bottlenecks, plant time-to-line-stop
Evidence panel (new)	Retrieved passages and series with source, date and trust tier; which claim each supports

View	Information
Scenario panel	Scenario weights and plan cost by scenario; tail risk
Plan view	Recommended reservations and routes; diff against the active plan
Computation trace	Operator sequence, cost, why deeper analysis was chosen (including retrieval and rewrite steps)
Verification panel	Each check, value, threshold, verdict
What-if	Change severity or duration, recompute a fast path
Human action	Approve / Edit / Reject, stored to decision memory

10. Additional improvements over v1.0

Improvement	Rationale	Tier
Canonical entity registry across all stores	Core integration mechanism; removes ID mismatches between events, documents, series and plans	1
Plant line-stop risk / time-to-line-stop KPI	Domain-specific outcome for just-in-time automotive; makes results interpretable to planners	1
PortWatch anomaly-triggered events	Lets time-series data originate events, not only text feeds; strengthens integration of perception with data	1–2
Real-event backtesting	Replays documented regional events (for example cyclone episodes affecting the Chennai coast) and compares predicted port impact with PortWatch port-call series, if coverage is confirmed	2
Planted-analogue synthetic benchmark	Makes the benefit of retrieval measurable rather than assumed	2
Evidence sufficiency gate and query rewriting	Prevents both over-retrieval and acting on weak evidence	1–2
Calibrated uncertainty	Confidence calibration (isotonic/temperature) and conformal intervals on propagation error; report ECE	2
Retrieval security (poisoning, staleness, injection)	Trust tiers, quarantine of retrieved text, red-team suite S3	1
Cost-aware LLM usage	Response and embedding caches; small model for grading; token cost in the controller objective	1–2
Tiered MVP	Tier 1: integrated pipeline, rule router, text+graph RAG, hop-decay, OR-Tools, verifier, dashboard. Tier 2: time-series retrieval, scenarios/CVaR, learned controller, backtests. Tier 3: GNN surrogate, DAgger, RoRo commodity	—
One-command reproducibility	Fixed seeds, JSONL traces, Docker image, single script regenerating all tables and figures	1

11. Evaluation plan

The project is evaluated as an integrated system, with ablations that remove one integration link at a time.

Question	Comparison	Primary metric
Does graph analysis improve impact identification?	No graph vs k-hop/exposure graph	Affected-path recall; exposure error

Question	Comparison	Primary metric
Does retrieval help?	No RAG vs fixed RAG vs agentic RAG	Regret at equal cost; cost at equal regret
Is the retrieval itself good?	Text-only vs graph-aware vs graph + time-series retrieval	Recall@k on planted analogues; context precision
Are explanations grounded?	With vs without citation check	Claim-citation support rate; unsupported-claim rate
Is the evidence-sufficiency gate calibrated?	Gate vs fixed number of retrievals	Calibration of sufficiency vs correct-analogue found; steps to answer
Does adaptive control save compute?	Always-full vs rule router vs LLM planner vs learned controller	Regret vs cost frontier; AUROC for depth prediction; ECE
Does propagation add value?	Hop-decay vs simulator (vs GNN if built)	Recall@10, error, runtime
Does scenario planning help?	Single-state vs scenario-aware	Tail regret / CVaR; unmet demand; line-stop events
Does verification work?	Injected plan faults and injected instructions	Fault catch rate; state unchanged
Does it work on real events?	Backtest on documented regional events	Qualitative agreement with PortWatch series; lead time; rank correlation of predicted vs observed port impact

11.1 Benchmark suites

- **S1** single-event core across network sizes and ambiguity tiers. **S2** timelines with updates, retractions and related reports.
- **S3** adversarial inputs: false and conflicting reports, prompt injection in articles **and in retrieved documents**, poisoned or stale chunks.
- **S4** mutated plans with injected constraint/objective faults. **S5** short live-feed episodes for qualitative demonstration.
- **S6 (new)** real-event backtests against PortWatch series. Networks are split by topology, not by event, so test networks are unseen in training.

11.2 Targets

Area	Target / success rule	Status
Adaptive computation	≥40% lower mean cost at comparable regret versus fixed-full routing	Target from methodology
Depth prediction	AUROC ≥ 0.80; ECE ≤ 0.08	Target from methodology
Propagation	Recall@10 ≥ 10 points above hop-decay; ≥10× faster than simulator (applies if GNN is built)	Target; Tier 3
Risk-aware planning	≥15% lower 95th-percentile regret at ≤5% higher mean cost	Target from methodology
Verification	≥99% injected-fault catch rate; no state change from injected instructions	Target from methodology
Retrieval value	Agentic RAG achieves lower regret than no-RAG on planted-analogue instances at ≤1.5× cost; retrieval Recall@5 on planted analogues ≥ 0.80	Proposed; tune after pilot

Area	Target / success rule	Status
Grounding	≥95% of released explanation claims supported by cited evidence	Proposed; tune after pilot
Demonstration	Live event → graph update → affected paths → agentic trace with retrieval → verified plan, end to end	Required

12. Technology stack and interfaces

Layer	Primary choice	Note
Language / validation	Python 3.11; Pydantic v2	Schemas at every boundary
Graph build and analysis	NetworkX; OSMnx for road/rail extraction	Graph database only if scale demands
Storage	SQLite; DuckDB for time-series	PostgreSQL if concurrent writers are needed
Retrieval	FAISS + sentence-transformers	Multilingual embedding model if regional-language news is used
Agent orchestration	Custom typed state machine (≈ a few hundred lines)	LangGraph or similar optional; not required
LLM	Hosted API or local model behind one interface	Deterministic mock for tests; response cache
Optimisation	Google OR-Tools; SciPy HiGHS cross-check	Affected-subgraph optimisation for speed
ML	PyTorch (masked MLP controller); PyTorch Geometric optional	GNN is Tier 3
Backend / frontend	FastAPI + Uvicorn; React + Cytoscape.js	Mocked API for early UI work
Testing / delivery	PyTest + Hypothesis; Docker; GitHub Actions	CI runs the integration test on every merge

12.1 API surface

Endpoint	Purpose
POST /events	Submit an event; return event ID and extraction record
GET /events/{id}	Event, cluster, confidence, lifecycle
POST /events/{id}/plan?lambda=	Run the agentic controller; return plan, verdict and trace ID
GET /plans/{id}	Reservations, scenario flows, costs, verification report
GET /traces/{id}	Operator sequence, cost, timing, state features, retrieved evidence IDs
GET /evidence/{id}	Retrieved item with source, date, trust tier and the claims it supports
GET /network	Current graph, multipliers, predicted impact and uncertainty
GET /registry	Canonical entity registry (IDs, names, aliases, geography)
POST /whatif	Change severity/multiplier and recompute a fast path
POST /decisions	Store human approve / edit / reject
POST /outcomes	Ingest realised outcomes (e.g. PortWatch series) to recalibrate the graph (loop L1)

Secrets (feed and LLM credentials) are environment variables or secret-store values; the repository contains no keys; logs hold source identifiers and request metadata only.

13. Team distribution and roadmap

13.1 Roles

Roles are organised around the **seams** of the integrated system rather than isolated modules, so that each member owns both a component and its interface to the next stage.

Member	Owns	Key responsibilities	Primary stack
M1 — Agent core & RAG lead	Stage C	Controller, baselines, grader, query rewriter, retrieval tools, explainer, oracle labels, LLM interface	Python, PyTorch, LLM API, FAISS
M2 — Graph & optimisation lead	Stage B + optimiser	Corridor graph, path engine, line-stop risk, propagation, scenarios, OR-Tools	NetworkX, OSMnx, OR-Tools, SciPy
M3 — Data & integration backbone	Stage A + USKL + API	Source adapters, PortWatch detector, entity registry, stores, ingestion of tagged documents, FastAPI, secrets	FastAPI, Pydantic, SQLite/DuckDB
M4 — Frontend & visualisation	Stage E	Corridor map, evidence panel, trace view, scenario and plan views, what-if	React, Cytoscape.js, TypeScript
M5 — QA, evaluation & delivery	Cross-cutting	Verifier tests, synthetic generator with planted analogues, backtest harness, metrics, CI/CD, reproducibility	PyTest, Hypothesis, Docker, GitHub Actions

- M1 + M2 jointly own the controller–graph feature boundary; M2 + M3 own graph and event contracts; M1 + M3 own the vector-memory schema and tagging.
- Weekly **integration build** on a fixed day: the end-to-end scripted event must pass before new features merge.
- All members own tests for their component and review pull requests touching shared schemas.

13.2 Fourteen-week roadmap (vertical slices)

Weeks	Milestone	Owners
1–2	Data-readiness check (gate G0); entity registry; corridor graph v0; walking skeleton with stubs and CI integration test	M2, M3, M5
3–4	Live adapters (GDACS, GDELT/RSS, PortWatch anomaly detector); extraction and entity resolution; vector memory ingestion with graph tags; curated regional case library	M1, M3
5–6	Path-impact engine and line-stop risk; text + graph RAG with grader; synthetic generator with planted analogues; verifier L1–L3	M1, M2, M5
7–8	Time-series retrieval; hop-decay and simulator propagation; scenarios; backtest harness; dashboard v1 with evidence panel	M2, M4, M5
9	Operator wrappers, cost profiling, agent executor, rule/LLM/fixed-RAG baselines	M1, M2, M3
10–11	Oracle labels, imitation policy, safety masks; GNN only if time remains	M1, M5
12	Full benchmark, ablations, backtests, statistical evaluation	M5, all
13	Live end-to-end integration, trace/evidence visualisation, red-team suite S3	M3, M4, M5
14	Freeze, one-command reproducibility run, report and demo	All

13.3 Development gates

Gate	Pass condition
G0 (Week 2)	Each dataset in Section 2.4 verified for access, licence and coverage, or its fallback chosen
G1 (Week 2)	Walking skeleton runs event → verified plan; CI integration test green
G2	Affected-path and line-stop outputs reproducible for fixed seeds; baseline optimisation has zero unintended violations
G3	Verifier catches injected faults in the mutation suite
G4	Controller emits only valid actions; trace includes retrieved evidence IDs; explanations pass citation check
G5	Held-out evaluation and backtests frozen; all results regenerate from one command

14. Risks and design decisions

Risk	Impact	Mitigation
Public data coarser or less available than assumed	Weak calibration; thin backtests	Gate G0; documented fallbacks; synthetic generator calibrated to aggregates; state limits openly
Synthetic-to-real gap	Results do not transfer	Backtests on real events; claim method-level results only
Live API instability / rate limits	Demo interruption	Cached events; analyst-input fallback; replayable episodes
LLM hallucination, prompt injection, poisoned documents	Wrong graph update or unsupported claim	No-tools extractor; untrusted-text handling; trust tiers; L1 and L3 verification; S3 suite
Agent loops or runaway cost	Slow or expensive runs	Step, tool and budget limits; hard masks; forced ESCALATE
Adaptive policy ties rule router	Weaker novelty claim	Treat router as real baseline; report ties
GNN underperforms heuristic	Weak research result	GNN is Tier 3; heuristic stays default
Scope creep	Schedule failure	Fixed industry and region; tiered MVP; one commodity
Integration delays	Team blocking	Walking skeleton Week 2; CI integration test; typed contracts; weekly integration build

Key design decisions

- One pipeline over one shared state; the entity registry is the glue. Modules are stages, not products.
- Scope is one industry and one region, chosen for data availability and domain sensitivity; no software-dependency graph is part of the system.
- Retrieval is agentic and computational: text, graph, time-series and decision memory are tools alongside analysis and optimisation.
- The LLM interprets, rewrites, grades and explains; deterministic optimisation decides feasibility; verification gates release.
- Adaptive computation is the research contribution; individual RAG, graph and optimisation techniques are established.
- Human review remains the default release mechanism; real feeds are demonstrations and synthetic data with backtests carries the evidence.

Appendix A — Unified state (excerpt)

```
{
  "event": {"id":"ev_0188","type":"HAZARD_PORT",
            "targets":[{"id":"PORT_KAMARAJAR","p":0.92}],
            "capacity_multiplier":0.40,"duration_days":3,"confidence":0.78},
  "graph": {"version":"g_2026-10-16","affected_paths":14,"flow_exposure":0.38,
            "plant_line_stop":{"PLANT_ORAGADAM":{"time_to_line_stop_days":2.1}}},
  "evidence": [{"id":"doc_0412","source":"rss","date":"2023-12-05",
                "trust":"B","entities":["PORT_CHENNAI"]}],
  "sufficiency": 0.82,
  "results": {"propagation":"art_p07","scenarios":"art_s03","plan":"plan_0091"},
  "budget": {"lambda":0.2,"steps":8,"cost_spent":6.4,"step_limit":12},
  "trace": ["GRAPH_ANALYZE","RETRIEVE_TEXT","REWRITE_QUERY","RETRIEVE_TS",
            "GRADE","PROPAGATE","SCENARIOS","OPTIMIZE","VERIFY","STOP"]
}
```

Appendix B — Agent loop (pseudocode)

```
state = perceive(event)                        # always runs; resolves to registry IDs
while not state.done:
    mask   = valid_operators(state)          # hard safety + precondition masks
    action = policy(features(state), lam, mask)
    if action == STOP:      break            # only valid after VERIFY == PASS
    if action == ESCALATE:  state = escalate(state); break
    state, result = TOOLS[action].run(state, cfg, rng)   # retrieve OR compute; cost measured
    if action in RETRIEVE_TOOLS:
        state = grade(state)                 # relevance per item + evidence sufficiency
        if state.sufficiency < GATE and state.rewrites < 2: state.force = REWRITE_QUERY
    if state.steps >= STEP_LIMIT or state.cost > BUDGET: state = escalate(state); break
release(state) if state.verify == PASS else hand_to_human(state)
```

Appendix C — Data sources to verify in Week 1–2

Source	Address	Check
IMF PortWatch	portwatch.imf.org (datasets also on data.humdata.org/organization/portwatch)	Indian port coverage, API access, update lag
GDACS	www.gdacs.org	Feed format, regional alert history
GDELT	www.gdeltproject.org	Query limits, India/logistics filters
OpenStreetMap	www.openstreetmap.org (OSMnx for extraction)	Road/rail completeness; licence and attribution
UN Comtrade	comtradeplus.un.org	Free-tier limits; HS codes for auto sector
India open data	data.gov.in; Ministry of Ports, Shipping & Waterways publications	Port traffic granularity and recency
Weather	open-meteo.com; mausam.imd.gov.in	Terms of use; programmatic access

The 39-page methodology report remains the authoritative implementation reference. This document consolidates it into a project-facing specification for the integrated, region-specific, agentic-RAG edition.

END OF DOCUMENT